

Summary of Strassen's Algorithm and Implementation Notes

Conceptual Overview

1. **Recursive Partitioning:** - Strassen's algorithm recursively divides matrices A and B into four quadrants:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

- This partitioning continues until reaching the base case of $N = 1$, where scalar multiplication is performed.

2. **Base Case:** - When $N = 1$, each quadrant is a single element of the original matrix, and the actual multiplication occurs:

$$C[0][0] = A[0][0] \times B[0][0]$$

- The recursion then unwinds, merging results from smaller matrices back up to form the final matrix C .

3. **Combining Results:** - The results of the multiplications are combined as the recursion unwinds:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

- This recombination reduces the number of multiplications from N^3 to $N^{\log_2 7} \approx N^{2.81}$.

4. **No Explicit Partitioning in Code:** - The algorithm conceptually partitions matrices, but in practice, no new matrices are created. - Pointers, indices, or views are used to reference submatrices, avoiding overhead from copying.

Implementation and Optimization Notes

1. **Zero-Based Indexing in Code:** - Instead of using A_{11} , A_{12} , etc., the submatrices can be accessed using zero-based indexing such as $A[0][0]$, $A[0][1]$, ...

2. **Custom Subscript Operator:** - Overloading the subscript operator can be useful for managing matrix partitioning. However, it should be evaluated for its trade-offs, as it might conflict with data-oriented design principles.

3. **Parallelization and SIMD:** - Given the recursive structure and power-of-2 dimensions, Strassen's algorithm has potential for optimization with SIMD instructions or multi-threading. - Consider using row-major order for better cache efficiency and easier SIMD vectorization.

4. **Base Case Optimization:** - The base case of $N = 1$ can be a critical bottleneck. Using intrinsic functions or hardware acceleration at this level could significantly improve performance.

5. **Understanding the Data Flow:** - Before implementing optimizations, it's essential to have a clear understanding of how the algorithm flows and how results are collated as the recursion unwinds. - The recurrence relations and the way intermediate products are added back into C are key to implementing the algorithm correctly.